

片段 7.1)。在从运行队列选择下一个需要调度的任务时，步幅调度会选取当前虚拟时间最小的任务，将该任务移出运行队列 `run_queue` (`remove_queue_min`, 第 4 行)，并最终返回该被选择的任务 (第 6 行)。

代码片段 7.3 步幅调度伪代码

```

1 struct process* sched_dequeue(void)
2 {
3     // 选择并移除运行队列中虚拟时间最小的任务
4     struct process* proc = remove_queue_min(run_queue);
5     // 调度该任务
6     return proc;
7 }
8
9 void sched_enqueue(struct process* proc)
10 {
11     // 基于该任务的步幅 stride, 计算调度后的虚拟时间 pass
12     proc->pass += proc->stride;
13     // 将该任务重新插入运行队列中
14     insert_queue(run_queue, proc);
15 }

```

当任务被执行完一个时间片，需要放回运行队列时，需要更新该任务所经历的虚拟时间。具体地，步幅调度为每个任务维护 `pass` 变量 (第 12 行)，表示该任务的虚拟时间。使用 `stride` 变量保存每个任务的步幅 (第 12 行)。在调度结束后按步幅增加其虚拟时间 (第 12 行)，并将该任务重新加入运行队列 `run_queue` 中 (`insert_queue`, 第 14 行)。为了使 `stride` 是整数，会设置一个极大的整数 `MaxStride` (图 7.15 为了直观，设置 `MaxStride` 为任务 A 和任务 B 的份额的最小公倍数)，并根据公式 $\text{stride} = \text{MaxStride} / \text{ticket}$ 计算 `stride`，其中 `ticket` 就是份额。

在真实系统中需要注意的是，由于任务可能在任意时间进入系统，因此任务的 `pass` 不能简单地从 0 开始设置，而应该设置为当前所有任务的最小 `pass` 值。否则，可能会导致新进入的任务长时间占用 CPU。

7.5.3 份额与优先级的比较

上文介绍了两种公平共享调度策略：彩票调度和步幅调度。彩票调度使用随机的思想，因此设计和实现会相对简单，但可能无法保证完全公平，特别是在任务运行时间较短、调度次数不足的情况下。相对地，步幅调度则完全保证了任务的公平共享，但代价是需要额外的维护开销。

此外，在优先级调度策略和公平共享调度策略中，优先级和份额都体现了任务的重要性，它们的差异直接导致了两种调度策略适用场景的不同。优先级与份额都表示任务在系统中的重要程度，但是它们的目的是不同的。优先级调度是为了优化任务的周转时

间、响应时间和资源利用率而设计的。不同任务的优先级只能用于相互比较，表明任务执行的先后。而基于份额的公平共享调度是为了让每个任务都能使用它应获得的系统资源。份额的值明确对应任务应使用的系统资源比例，而不同任务的份额可以用于计算这一比例。因此，在优先级调度策略和公平共享调度策略中进行选择时，必须明确当前目标场景更适合优先级还是份额。

7.6 多核处理器调度机制

本节主要知识点

- ❑ 如何维护多核处理器的运行队列？
- ❑ 如何均衡多核处理器的负载？
- ❑ 如何指定运行某个任务的 CPU 核心？

在之前的章节中，我们主要讨论了单核场景下的处理器调度策略。在该场景下，调度器只需要选择合适的任务让唯一的 CPU 核心执行即可。然而，随着多核处理器的出现，处理器调度也变得更加复杂。在多核场景下，调度器不仅需要选取合适的任务，还需要分配合适的 CPU 核心用于执行任务。为了支持多核处理器调度，调度器机制也要进行对应的扩展，本节将主要介绍多核处理器调度的机制。由于篇幅原因，更多有关多核处理器调度的内容将在本书的在线部分进行介绍。

7.6.1 运行队列

一种扩展单核处理器调度机制的直观思路是让所有 CPU 核心共享一个全局的运行队列。图 7.16 展示了基于全局运行队列的多核调度机制。当一个 CPU 核心发起调度时，根据给定的调度策略，从全局运行队列中选择下一个由它执行的任务。当任务的时间片耗尽后，任务会被重新放回全局运行队列中。

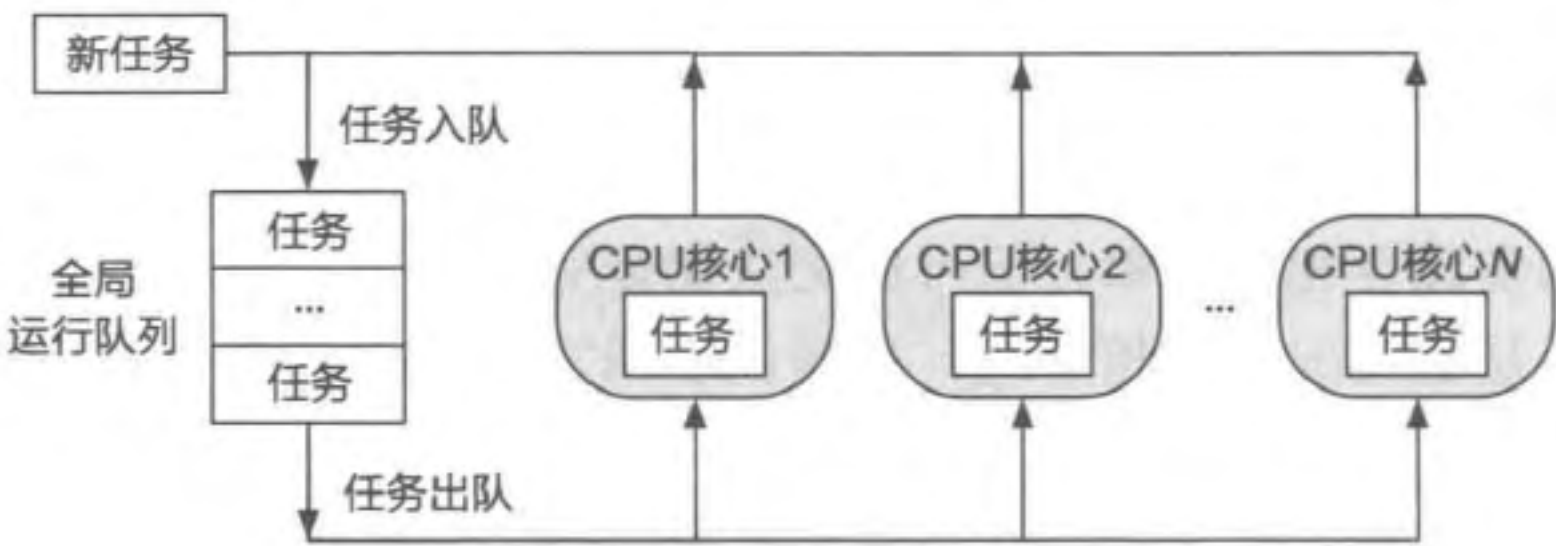


图 7.16 基于全局运行队列的多核处理器调度

全局运行队列是一种简单直观的多核调度机制，但也会导致许多性能方面的问题。

首先,多个 CPU 核心对全局运行队列的访问会产生锁竞争,随着 CPU 核心数量的上升,任务调度的开销会越来越大。另一方面,系统中的任务会在不同的 CPU 核心间来回切换,导致无法有效利用每个核心上的高速缓存 (CPU cache),并对任务的执行效率产生影响。

为了避免上述问题,现代操作系统常用的方式是让每个 CPU 核心维护一个本地的运行队列。如图 7.17 所示,当新任务到达系统时,会被分配到某一个 CPU 核心的本地队列,参与该 CPU 核心的单核调度。由于 CPU 核心在调度任务时只需要访问本地数据,且通常情况下任务不会被频繁跨核迁移,因此能够保证高效的任务调度与执行。

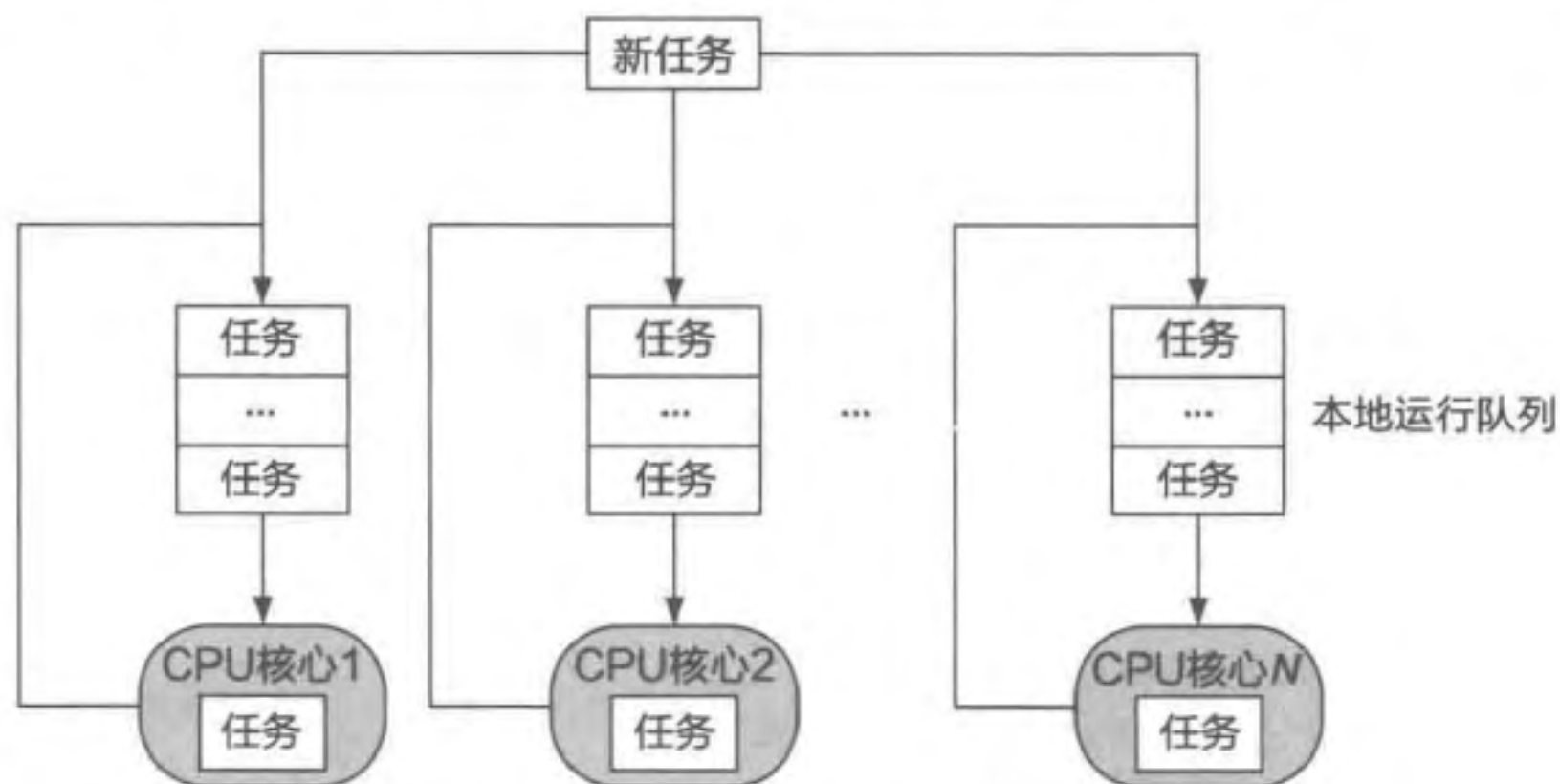


图 7.17 基于本地运行队列的多核处理器调度

调度器通过将任务放到 CPU 核心的本地运行队列,避免了任务在多核间切换的性能开销,因而在多核调度时有良好的性能。然而,如果任务在它的生命周期中只在一个 CPU 核心上运行,则可能导致多核间的负载不均衡,例如某个 CPU 核心的利用率为 100%,而剩余 CPU 核心的利用率都为 0%。为了解决这一问题,多核处理器调度必须考虑负载均衡,通过追踪每个 CPU 核心当前的负载情况,将高负载 CPU 核心管理的任务迁移到低负载 CPU 核心上,尽可能地保证每个核心的负载大致相同。

7.6.2 负载均衡与负载追踪

为了达到负载均衡的目的,首先要明确系统中的负载是什么。一种简单的负载定义方式是,将每个 CPU 核心本地队列的长度作为负载。那么负载均衡就应尽可能地让所有 CPU 核心本地队列的长度保持均匀。对于负载的不同定义,可能会引申出不同的负载均衡策略,7.7.4 节将进一步展开介绍 Linux 是如何对负载进行定义,从而指导负载追踪与负载均衡的。

当某个 CPU 核心发现自己本地队列的任务全部执行完时,可以从其他 CPU 核心的本地队列中拿取等待中的任务执行,保证不会有 CPU 核心处于空闲状态,并最终达到负载均衡的目的。这种空闲 CPU 核心从其他核心窃取任务执行的方式也被称为工作窃取 (Work Stealing)。

7.6.3 处理器亲和性

在多核调度中，操作系统能够根据特定策略在 CPU 核心间迁移任务，从而达到负载均衡的目的。然而，有时候开发者并不希望调度器这么做。例如，小明现在已经是一位有经验的开发者，他完全熟悉自己的程序和调度器的设计。他希望自己程序的任务只在他指定的 CPU 核心上执行。

为了使开发者能够控制自己程序的执行，Linux 和 ChCore 等操作系统都提供了处理器亲和性（Processor Affinity）的机制，允许程序对任务可以使用的 CPU 核心进行配置。任务可设置 `cpu_set_t` 掩码，用于表示可以执行任务（以线程为单位）的 CPU 核心集合，掩码中的每一位都对应一个 CPU 核心。如代码片段 7.4 所示，提供了一系列对 `cpu_set_t` 进行操作的代码宏。

代码片段 7.4 操作 CPU 核心集合的代码宏

```

1 #include <sched.h>
2
3 // 对 set 进行初始化，设置为空
4 void CPU_ZERO(cpu_set_t *set);
5 // 将对应 CPU 核心加入 set 中
6 void CPU_SET(int cpu, cpu_set_t *set);
7 // 将对应 CPU 核心从 set 中删除
8 void CPU_CLR(int cpu, cpu_set_t *set);
9 // 判断对应 CPU 核心是否在 set 中，是则返回 1，否则返回 0
10 int CPU_ISSET(int cpu, cpu_set_t *set);
11 // 返回当前 set 中的 CPU 核心数量
12 int CPU_COUNT(cpu_set_t *set);
13 ...

```

在这些操作宏的基础上，操作系统提供了设置和获取任务亲和性的系统调用，如代码片段 7.5 所示。程序可以通过 `sched_setaffinity` 设置任务对于 CPU 核心的亲和性，或是通过 `sched_getaffinity` 获取任务的亲和性配置。需要注意的是，由于线程是 Linux 的调度单元，这里的 `pid` 参数实际上是 Linux 为每个线程分配的标识符，如果 `pid` 为 0，代表操作对象是当前的调用者线程。

代码片段 7.5 Linux 的任务（线程）亲和性接口

```

1 #include <sched.h>
2
3 int sched_setaffinity(pid_t pid, size_t cpusetsize,
4                       const cpu_set_t *mask);
5 int sched_getaffinity(pid_t pid, size_t cpusetsize,
6                       cpu_set_t *mask);

```

代码片段 7.6 展示了设置任务亲和性的示例代码。设置完成后，该任务只能在 CPU 核心 0 和 2 上执行。操作系统在进行任务调度和迁移时，会检查其迁移目标是否在该任

务允许使用的 CPU 核心集合（包含 0 和 2 两个 CPU 核心）中，如果不在，则不会迁移任务。

代码片段 7.6 设置线程亲和性的代码示例

```
1 #include <sched.h>
2 #include <stdio.h>
3 #include <sys/sysinfo.h>
4
5 int main(int argc, char *argv)
6 {
7     cpu_set_t mask;
8     // 初始化 mask 的 CPU 集合为空
9     CPU_ZERO(&mask);
10
11     // 在 mask 的 CPU 集合中加入 CPU 核心 0 和 CPU 核心 2
12     CPU_SET(0, &mask);
13     CPU_SET(2, &mask);
14
15     // 根据 mask 设置当前线程的亲和性
16     sched_setaffinity(0, sizeof(mask), &mask);
17
18     // 执行程序逻辑
19     ...
20
21     return 0;
22 }
```

7.7 案例分析：Linux 调度器

本节通过案例分析的方式介绍 Linux 发展历史上使用过或正在使用的调度器与相关机制。Linux 诞生伊始，开发重点并非处理器调度。当时对调度器的要求是：实现简单，确保任务不会饥饿。因此，Linux 在 1.2 版本时使用了时间片轮转策略，将一个全局的环形队列作为调度器的运行队列。后来，随着计算机的处理能力变强，以及对称多处理架构（Symmetric Multi-Processor, SMP）的出现，用户将越来越多的任务交由 Linux 处理，处理器调度面对的工作场景越来越复杂。时间片轮转策略仅仅考虑了任务的执行时间，无法满足复杂场景所带来的日益严苛的调度需求。

具体来说，Linux 调度器需要考虑非常多的因素（以及它们对应的需求），包括但不限于：

- 任务的种类。Linux 将任务分为实时任务、交互式任务和批处理任务，并且调度器必须保证实时任务优先于其他两类任务执行。
- 任务使用的资源。根据不同任务使用的主要资源的不同，可以粗略地将任务分类为计算密集型任务和 I/O 密集型任务。为了尽可能保证计算机整体的资源利用率，